Tuần 1: Foundation (Day 1–7)

Day 1–2: DB Schema

6 bảng cốt lõi:

```
-- Knowledge Components
CREATE TABLE knowledge_components (
  id UUID PRIMARY KEY, code VARCHAR UNIQUE, name TEXT, grade INT, metadata JSONB
);

-- KST graph edges (A → B = A là prerequisite của B)
CREATE TABLE kc_prerequisites (
  kc_id UUID REFERENCES knowledge_components, prereq_id UUID REFERENCES knowledge_components,
  PRIMARY KEY (kc_id, prereq_id)
);
```

```
-- Items với IRT parameters
CREATE TABLE items (
  id UUID PRIMARY KEY, kc_id UUID REFERENCES knowledge_components,
  content JSONB,
  irt_a FLOAT DEFAULT 1.0,   -- discrimination
  irt_b FLOAT DEFAULT 0.0,   -- difficulty (b=0 là medium)
  irt_c FLOAT DEFAULT 0.25   -- guessing (MCQ 4 options)
);

-- Student IRT profile (ability θ)
CREATE TABLE student_irt (
  student_id UUID PRIMARY KEY, theta FLOAT DEFAULT 0.0, theta_se FLOAT DEFAULT 1.0
);

-- BKT state per student per KC
CREATE TABLE student_kc (
  student_id UUID, kc_id UUID,
  p_mastery FLOAT DEFAULT 0.1,   -- P(L_n) — cập nhật sau mỗi response
  p_know0   FLOAT DEFAULT 0.1,   -- P(L_0)
  p_transit FLOAT DEFAULT 0.30,  -- P(T)
  p_guess   FLOAT DEFAULT 0.25,  -- P(G)
  p_slip    FLOAT DEFAULT 0.10,  -- P(S)
  is_mastered BOOLEAN DEFAULT FALSE,
  stability FLOAT DEFAULT 1.0,   -- cho forgetting curve
  last_practiced TIMESTAMP,
  PRIMARY KEY (student_id, kc_id)
);

-- Response history
CREATE TABLE responses (
  id UUID PRIMARY KEY, student_id UUID, item_id UUID, kc_id UUID,
  correct BOOLEAN, context VARCHAR(20), -- 'assessment' | 'practice' | 'review'
  created_at TIMESTAMP DEFAULT NOW()
);

Day 3–4: Knowledge Graph (KST)
import networkx as nx

class KnowledgeGraph:
    def __init__(self):
        self.G = nx.DiGraph()   # edge A→B = A là prerequisite của B

    def load_from_db(self, db):
```

```
    for kc in db.all_kcs():
        self.G.add_node(kc['id'], **kc)
    for edge in db.all_prerequisites():
        self.G.add_edge(edge['prereq_id'], edge['kc_id'])

def find_starting_kc(self, known_kcs: set) -> str | None:
    """
    KST: Tìm KC 'trung tâm' nhất để bắt đầu assessment.

    Eligible KCs = KCs mà TẤT CẢ direct prerequisites đã trong known_kcs.
    Score = len(fulfilled ancestors) + len(unmastered descendants)
    → Maximize: chọn KC nằm ở vị trí giữa nhất của graph.

    New student (known_kcs = ∅): chỉ root KCs eligible,
    score = 0 + len(descendants) → chọn KC quan trọng nhất (nhiều block nhất).
    """
    best_kc, best_score = None, -1
    for kc in self.G.nodes():
        if kc in known_kcs:
            continue
        direct_prereqs = list(self.G.predecessors(kc))
        if not all(p in known_kcs for p in direct_prereqs):
            continue   # Chưa đủ prerequisites → bỏ qua

        all_ancestors = nx.ancestors(self.G, kc)
        all_descendants = nx.descendants(self.G, kc)
        score = (sum(1 for a in all_ancestors if a in known_kcs) +
                 sum(1 for d in all_descendants if d not in known_kcs))
        if score > best_score:
            best_score, best_kc = score, kc
    return best_kc

def navigate(self, current_kc: str, correct: bool,
             known_kcs: set) -> str | None:
    """
    KST navigation sau mỗi KC decision:
    - Pass (correct) → tìm successor mà tất cả prereqs đã fulfilled
    - Fail (wrong) → tìm prerequisite chưa master, ưu tiên quan trọng nhất
    """
    if correct:
        updated = known_kcs | {current_kc}
        candidates = [
            s for s in self.G.successors(current_kc)
            if s not in updated
```

```
            and all(p in updated for p in self.G.predecessors(s))
        ]
    else:
        candidates = [p for p in self.G.predecessors(current_kc)
                      if p not in known_kcs]

    if not candidates:
        return None
    # Ưu tiên KC có nhiều descendants nhất (quan trọng nhất)
    return max(candidates, key=lambda k: len(nx.descendants(self.G, k)))
```

Day 5–7: IRT Engine (3PL)

```
import numpy as np
from scipy.optimize import minimize_scalar
from typing import List, Tuple

class IRTEngine:
    """
    3-Parameter Logistic: P(X=1|θ) = c + (1-c) / [1 + exp(-Da(θ-b))]
    D=1.7 là scaling constant chuẩn (logistic ≈ normal ogive)
    """
    D = 1.7

    @classmethod
    def p_correct(cls, theta: float, a: float, b: float, c: float) -> float:
        return c + (1-c) / (1 + np.exp(-cls.D * a * (theta - b)))

    @classmethod
    def information(cls, theta: float, a: float, b: float, c: float) -> float:
        """Fisher information — đo lượng thông tin item cung cấp về θ"""
        p = cls.p_correct(theta, a, b, c)
        return (cls.D**2 * a**2 * (p-c)**2 * (1-p)) / ((1-c)**2 * p + 1e-10)

    @classmethod
    def update_theta(cls,
                     responses: List[Tuple[bool, float, float, float]],
                     init: float = 0.0) -> Tuple[float, float]:
        """
        MLE estimate of θ từ toàn bộ response history.
        responses = [(correct, a, b, c), ...]
        Trả về: (theta_hat, standard_error)
        SE = 1/√(ΣI(θ)) — càng nhiều item → SE càng nhỏ → estimate càng chính xác.
        """
        def neg_ll(theta):
```

```
        ll = 0
        for correct, a, b, c in responses:
            p = np.clip(cls.p_correct(theta, a, b, c), 1e-9, 1-1e-9)
            ll += correct * np.log(p) + (1-correct) * np.log(1-p)
        return -ll

    res = minimize_scalar(neg_ll, bounds=(-4, 4), method='bounded')
    theta_hat = res.x
    total_info = sum(cls.information(theta_hat, a, b, c)
                     for _, a, b, c in responses)
    return theta_hat, 1/np.sqrt(max(total_info, 0.01))

@classmethod
def select_zpd(cls, theta: float, items: list,
               target_p: float = 0.65) -> dict:
    """
    Zone of Proximal Development: chọn item sao cho P(correct|θ) ≈ target_p.
    Default 65%: đủ thách thức nhưng không gây frustration.
    Item difficulty b ≈ theta - log((1-c)/(target_p-c) - 1) / (D*a)
    """
    return min(items, key=lambda item:
        abs(cls.p_correct(theta, item['a'], item['b'], item['c']) - target_p))
```

Tuần 2: Assessment Engine (Day 8–14)

Day 8–11: CAT Controller

```
class CATController:
    """
    KST role: điều hướng DỌC GRAPH — KC nào hỏi tiếp
    IRT role: điều chỉnh ĐỘ KHÓ — câu nào hỏi trong KC đó
    """
    PASS_STREAK = 3     # 3 đúng liên tiếp → pass KC
    FAIL_STREAK = 3     # 3 sai liên tiếp → fail KC
    SE_STOP = 0.30      # θ ổn định đủ → có thể dừng
    MAX_ITEMS = 40

    def start(self, student_id: str) -> dict:
        known = db.get_known_kcs(student_id)
        start_kc = kg.find_starting_kc(known)
        theta = db.get_theta(student_id) or 0.0   # ω=0 cho new student

        items = db.get_items(start_kc)
        first_item = irt.select_zpd(theta, items, target_p=0.65)
```

```
    session = {
        'kc': start_kc, 'theta': theta, 'se': 1.0,
        'responses': [],      # list of (correct, a, b, c)
        'kc_results': {},     # {kc_id: 'pass'|'fail'|'gap'}
        'known': set(known),
        'streak_c': 0, 'streak_w': 0, 'n': 0
    }
    return {'session': session, 'item': first_item}

def respond(self, session: dict, item: dict, correct: bool) -> dict:
    s = session
    s['responses'].append((correct, item['a'], item['b'], item['c']))
    s['n'] += 1
    s['streak_c'] = (s['streak_c'] + 1) if correct else 0
    s['streak_w'] = (s['streak_w'] + 1) if not correct else 0

    # Update θ (MLE) — cần ít nhất 2 data points
    if len(s['responses']) >= 2:
        s['theta'], s['se'] = irt.update_theta(s['responses'], s['theta'])

    # Pass/fail decision
    if s['streak_c'] >= self.PASS_STREAK:
        return self._pass(s)
    if s['streak_w'] >= self.FAIL_STREAK:
        return self._fail(s)

    # Early stop: θ đủ chính xác VÀ đủ số câu
    if s['se'] < self.SE_STOP and s['n'] >= 10:
        avg_b = np.mean([i['b'] for i in db.get_items(s['kc'])])
        return self._pass(s) if s['theta'] > avg_b else self._fail(s)

    if s['n'] >= self.MAX_ITEMS:
        return self._finalize(s)

    items = db.get_items(s['kc'], exclude_seen=True)
    return {'status': 'continue', 'item': irt.select_zpd(s['theta'], items), 'session': s}

def _pass(self, s):
    s['kc_results'][s['kc']] = 'pass'
    s['known'].add(s['kc'])
    s['streak_c'] = s['streak_w'] = 0
    next_kc = kg.navigate(s['kc'], True, s['known'])
    if next_kc:
        s['kc'] = next_kc
```

```
            return {'status': 'continue',
                    'item': irt.select_zpd(s['theta'], db.get_items(next_kc)), 'session': s}
        return self._finalize(s)

    def _fail(self, s):
        s['kc_results'][s['kc']] = 'fail'
        s['streak_c'] = s['streak_w'] = 0
        next_kc = kg.navigate(s['kc'], False, s['known'])
        if next_kc:
            s['kc'] = next_kc
            return {'status': 'continue',
                    'item': irt.select_zpd(s['theta'], db.get_items(next_kc)), 'session': s}
        s['kc_results'][s['kc']] = 'fundamental_gap'
        return self._finalize(s)

    def _finalize(self, s):
        gaps = [k for k, v in s['kc_results'].items() if v != 'pass']
        return {
            'status': 'done', 'theta': s['theta'], 'theta_se': s['se'],
            'gaps': gaps,
            'mastered': [k for k, v in s['kc_results'].items() if v == 'pass'],
            'first_learning_kc': gaps[0] if gaps else None
        }
```

Day 12–14: Edge case testing

```
# 4 test cases bắt buộc phải pass:

def test_new_student_goes_to_foundation():
    # Student không biết gì → phải xuống roots
    session = cat.start('new_student')
    for _ in range(9):  # 3 sai liên tiếp × nhiều KC
        session = cat.respond(session['session'], session['item'], correct=False)
    assert 'fundamental_gap' in session['session']['kc_results'].values()

def test_expert_student_reaches_top():
    # Student biết tất cả → phải đi lên đến leaf nodes
    ...

def test_theta_converges():
    # Sau 20 responses, SE phải < 0.5
    responses = [(True, 1.0, 0.0, 0.25)] * 10 + [(False, 1.0, 0.0, 0.25)] * 10
    theta, se = IRTEngine.update_theta(responses)
    assert se < 0.5
```

```
def test_no_graph_cycles():
    # KST graph phải là DAG
    assert nx.is_directed_acyclic_graph(kg.G)
```

Tuần 3: Learning Loop (Day 15–21)

Day 15–17: BKT Engine

```
from dataclasses import dataclass

@dataclass
class BKTState:
    p_mastery: float       # P(L_n) — updated sau mỗi observation
    p_know0:   float = 0.10
    p_transit: float = 0.30
    p_guess:   float = 0.25
    p_slip:    float = 0.10

class BKTEngine:
    THRESHOLD = 0.95   # Tunable per KC

    def update_observation(self, s: BKTState, correct: bool) -> BKTState:
        """
        Corbett & Anderson (1994) standard BKT:

        P(correct)   = P(L)·(1-P(S)) + (1-P(L))·P(G)
        P(L|correct) = P(L)·(1-P(S)) / P(correct)          ← Bayesian update
        P(L|wrong)   = P(L)·P(S)     / P(wrong)
        P(L_{n+1})   = P(L_n|obs) + (1 - P(L_n|obs))·P(T)  ← Transition
        """
        p = s.p_mastery
        p_c = p * (1-s.p_slip) + (1-p) * s.p_guess
        p_w = p * s.p_slip    + (1-p) * (1-s.p_guess)

        p_post = (p*(1-s.p_slip)/p_c) if correct else (p*s.p_slip/p_w)
        p_next = p_post + (1-p_post) * s.p_transit

        return BKTState(np.clip(p_next, 0.01, 0.999), s.p_know0,
                        s.p_transit, s.p_guess, s.p_slip)

    def update_learning_event(self, s: BKTState,
                              p_transit_content: float = 0.70) -> BKTState:
        """
        Sau khi xem video/bài giảng:
```

```
        P_new = P_old + (1 - P_old) × P(T_content)

        Ví dụ từ spec:
        P_old = 0.05, P(T_video) = 0.85
        → P_new = 0.05 + 0.95 × 0.85 = 0.8575 ≈ 85% ✓

        Các content type có P(T) khác nhau:
        - Video giải chi tiết: 0.75–0.85
        - Bài đọc ngắn: 0.50–0.65
        - Worked example: 0.60–0.75
        """
        p_new = s.p_mastery + (1-s.p_mastery) * p_transit_content
        return BKTState(np.clip(p_new, 0.01, 0.999), s.p_know0,
                        s.p_transit, s.p_guess, s.p_slip)

    def is_mastered(self, s: BKTState) -> bool:
        return s.p_mastery >= self.THRESHOLD
```

Day 18–19: IRT–BKT Integration (phần lý thuyết quan trọng nhất)

Như trong mô tả: **θ (IRT ability) và P(L) (BKT mastery) là 2 dimension độc lập.** θ ảnh hưởng BKT theo 3 kênh:

```
def init_bkt_with_irt(kc_id: str, theta: float) -> BKTState:
    """
    IRT theta điều chỉnh BKT params KHI KHỞI TẠO KC mới.

    KÊNH 1: θ → P(L0) cao hơn cho HS giỏi
      "Dạy A, HS giỏi hiểu ngay A+1 → P(L0) của A+1 tự động cao"
      sigmoid(θ - 0.5) scaled [0.05, 0.45]

    KÊNH 2: θ → P(S) thấp hơn (ít bất cẩn hơn)
      HS giỏi ít mắc lỗi careless → P(S) giảm theo θ

    KÊNH 3: θ → P(T) cao hơn (học nhanh hơn)
      HS giỏi absorb content nhanh hơn

    KHÔNG thay đổi: P(G) — xác suất đoán mò không phụ thuộc vào θ,
    mà phụ thuộc vào loại câu hỏi (MCQ 4 options → P(G)=0.25 luôn).
    IRT kiểm soát P(G) bằng cách chọn câu KHÓ hơn cho HS giỏi
    (câu khó hơn → P(G) hiệu dụng thấp hơn, vì HS khó đoán đúng hơn).
    """
    base = db.get_kc_params(kc_id)   # default BKT params cho KC này

    # Kênh 1: P(L0) tăng theo θ
```

```
    p_know0 = 0.05 + 0.40 / (1 + np.exp(-(theta - 0.5)))

    # Kênh 2: P(S) giảm theo θ (bounded)
    p_slip = max(0.04, base['p_slip'] - 0.015 * np.clip(theta, -2, 2))

    # Kênh 3: P(T) tăng nhẹ theo θ
    p_transit = min(0.85, base['p_transit'] + 0.04 * np.clip(theta, -2, 2))

    return BKTState(
        p_mastery=p_know0,     # Ban đầu = P(L0)
        p_know0=p_know0,
        p_transit=p_transit,
        p_guess=base['p_guess'],   # P(G) không đổi — IRT kiểm soát
        p_slip=p_slip
    )


def pick_practice_item(theta: float, p_mastery: float, items: list) -> dict:
    """
    Từ spec: "HS có thể master KC với 6 câu dễ" — câu khó KHÔNG giúp BKT
    tăng P(T) hơn. Câu khó ảnh hưởng qua P(G)/P(S) hiệu dụng.

    → Điều chỉnh ZPD target theo learning stage:
      - Early (P(L) < 0.4): target 75% — build confidence, reduce frustration
      - Mid (0.4 ≤ P(L) < 0.75): target 65% — optimal challenge zone
      - Late (P(L) ≥ 0.75): target 55% — harder items để confirm mastery,
        làm giảm P(G) hiệu dụng (HS không thể đoán được → BKT tin P(L) cao hơn)
    """
    if p_mastery < 0.40:   target = 0.75
    elif p_mastery < 0.75: target = 0.65
    else:                  target = 0.55
    return IRTEngine.select_zpd(theta, items, target_p=target)
```

Day 20–21: Forgetting Curve + Learning Controller

```
class ForgettingCurve:
    """
    Exponential decay: P(t) = (P_mastered - floor)·exp(-λ/stability·t) + floor
    Khi P(t) < threshold → trigger review session
    """
    λ = 0.10              # decay rate per day
    THRESHOLD = 0.80      # trigger review khi xuống 80%
    FLOOR = 0.25

    def p_at_time(self, p0: float, days: float, stability: float = 1.0) -> float:
```

```
        return (p0 - self.FLOOR) * np.exp(-self.λ/stability * days) + self.FLOOR

    def days_until_review(self, p0: float, stability: float = 1.0) -> float:
        if p0 <= self.THRESHOLD: return 0
        return -(stability/self.λ) * np.log(
            (self.THRESHOLD - self.FLOOR) / (p0 - self.FLOOR)
        )

    def update_stability(self, s: float, correct: bool) -> float:
        return s * 1.5 if correct else max(0.5, s * 0.5)


class LearningController:
    def start_kc(self, student_id: str, kc_id: str) -> dict:
        theta = db.get_theta(student_id)
        state = init_bkt_with_irt(kc_id, theta)   # IRT → BKT injection
        db.save_bkt(student_id, kc_id, state)
        return {'content': db.get_content(kc_id), 'p': round(state.p_mastery, 3)}

    def after_content(self, student_id: str, kc_id: str, p_t: float = 0.70) -> dict:
        state = bkt.update_learning_event(db.get_bkt(student_id, kc_id), p_t)
        db.save_bkt(student_id, kc_id, state)
        theta = db.get_theta(student_id)
        item = pick_practice_item(theta, state.p_mastery, db.get_items(kc_id))
        return {'item': item, 'p': round(state.p_mastery, 3)}

    def after_practice(self, student_id: str, kc_id: str,
                       item: dict, correct: bool) -> dict:
        state  = bkt.update_observation(db.get_bkt(student_id, kc_id), correct)
        hist   = db.get_responses(student_id) + [(correct, item['a'],item['b'],item['c'])]
        new_θ, new_se = IRTEngine.update_theta(hist)

        db.save_bkt(student_id, kc_id, state)
        db.save_theta(student_id, new_θ, new_se)

        if bkt.is_mastered(state):
            days = fc.days_until_review(state.p_mastery)
            db.schedule_review(student_id, kc_id, days)
            db.mark_mastered(student_id, kc_id)
            known = db.get_known_kcs(student_id)
            next_kc = kg.navigate(kc_id, True, known)
            if next_kc:
                db.save_bkt(student_id, next_kc,
                            init_bkt_with_irt(next_kc, new_θ))  # inject ngay
```

```
            return {'status': 'mastered', 'next_kc': next_kc,
                    'review_in': round(days), 'p': round(state.p_mastery, 3)}

        item = pick_practice_item(new_θ, state.p_mastery, db.get_items(kc_id))
        return {'status': 'practice', 'item': item, 'p': round(state.p_mastery, 3)}
```

Tuần 4: Integration (Day 22–30)

Day 22–24: FastAPI endpoints

```
@app.post("/assessment/start/{sid}")
async def start_assessment(sid: str):
    return cat.start(sid)

@app.post("/assessment/respond/{sid}")
async def respond_assessment(sid: str, body: ResponseBody):
    return cat.respond(body.session, body.item, body.correct)

@app.post("/learn/start/{sid}/{kc}")
async def start_learning(sid: str, kc: str):
    return lc.start_kc(sid, kc)

@app.post("/learn/content/{sid}/{kc}")
async def content_viewed(sid: str, kc: str, body: ContentBody):
    return lc.after_content(sid, kc, body.p_transit)

@app.post("/learn/practice/{sid}/{kc}")
async def submit_practice(sid: str, kc: str, body: PracticeBody):
    return lc.after_practice(sid, kc, body.item, body.correct)

@app.get("/review/due/{sid}")
async def get_due(sid: str):
    mastered = db.get_mastered_kcs(sid)
    due = []
    for kc in mastered:
        days = (now() - kc['last_practiced']).days
        p_now = fc.p_at_time(kc['p_at_mastery'], days, kc['stability'])
        if p_now < fc.THRESHOLD:
            due.append({**kc, 'p_now': round(p_now, 3)})
    return sorted(due, key=lambda x: x['p_now'])   # urgency order
```

Day 25–27: React frontend (5 screens)

Screen	Key components
Assessment	Câu hỏi + θ estimate bar (hiển thị cho admin, ẩn với HS)
Learning	Video player + BKT progress arc (0→95%) + practice item
Mastery	Celebration + "ôn lại sau X ngày"
Dashboard	KST graph (react-force-graph) + review queue (due today/this week)
Review	Spaced repetition queue, sorted by urgency

Day 28–30: Calibration và params mặc định

IRT khi chưa có data (cold start):

Param	Default	Lý do
`a`	1.0	Discrimination trung bình
`b`	0.0	Medium difficulty (ω=0)
`c`	0.25	MCQ 4 options: 1/4
θ_0	0.0	Ability trung bình

Sau khi có 50+ responses/item → refit `a`, `b`, `c` bằng MML-EM (`py-irt` library).

BKT khi chưa có data:

Param	Default	Adjust range
$P(L_0)$	0.10	0.05–0.40 theo KC và θ
P(T)	0.30	0.20–0.85 theo KC type + θ
P(G)	0.25	Fixed theo format câu hỏi
P(S)	0.10	0.04–0.15 theo θ

Sau khi có data → dùng EM để fit params per KC:

from scipy.optimize import minimize

```
def fit_bkt_params(responses: list) -> dict:
    """EM-style MLE để fit P(T), P(G), P(S) cho từng KC"""
    def neg_ll(params):
        p_l0, p_t, p_g, p_s = np.clip(params, 0.01, 0.99)
        ll = 0
        for student_responses in responses:
            p_l = p_l0
            for correct in student_responses:
                p_obs = p_l*(1-p_s) + (1-p_l)*p_g
                ll += np.log(p_obs if correct else 1-p_obs + 1e-9)
                p_post = p_l*(1-p_s)/p_obs if correct else p_l*p_s/(1-p_obs)
                p_l = p_post + (1-p_post)*p_t
        return -ll

    res = minimize(neg_ll, x0=[0.1, 0.3, 0.25, 0.1],
                   bounds=[(0.01,0.5),(0.01,0.99),(0.01,0.5),(0.01,0.3)])
    p_l0, p_t, p_g, p_s = res.x
    return {'p_know0': p_l0, 'p_transit': p_t, 'p_guess': p_g, 'p_slip': p_s}
```

Notes triển khai quan trọng

Tech stack tối giản cho 1 tháng: Python (FastAPI) + PostgreSQL + Redis (session cache) + React + NetworkX + NumPy/SciPy. Tránh over-engineer ở giai đoạn này.

Về KST graph: cần ít nhất 20–30 KCs cho 1 unit để hệ thống có đủ depth để traverse. Validate bằng `nx.is_directed_acyclic_graph(G)` — nếu có cycle thì KST navigation sẽ bị loop vô hạn.

Về IRT cold start: với item chưa calibrate, dùng default `a=1.0, b=0.0, c=0.25`. Kết quả ZPD sẽ không perfect nhưng đủ tốt để demo. Sau khi có data, chạy MML-EM batch calibration qua đêm.

Về BKT threshold (0.95): có thể lower xuống 0.90 cho giai đoạn đầu để học sinh cảm giác tiến bộ nhanh hơn. Tune theo domain và age group.